



AUDIT REPORT

February 2026

For



CHERRY
ON FINANCE

Table of Content

Executive Summary	04
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	08
Techniques and Methods	10
Types of Severity	12
Types of Issues	13
Severity Matrix	14
■ Critical Severity Issues	15
1. Precision loss in position estimation reduces capital efficiency and distorts delta neutrality	15
2. Arithmetic underflow in delta calculation enables denial of service	17
■ High Severity Issues	20
3. Zero slippage protection in liquidity operations causes protocol insolvency through sandwich attacks	20
4. Dangerous slippage bypass for small amounts exposes users to MEV attacks	22
■ Medium Severity Issues	23
5. Excessive rebalance threshold allows dangerous directional exposure	23
6. Missing validation of oracle price staleness allows stale data usage	24
7. Missing strategy status validation allows deposits into disabled strategies	25
8. Missing slippage protection allows LP token value manipulation during deposits	26
9. Immutable supply cap prevents protocol scaling beyond initial limit	27
10. Unsafe token transfer prevents compatibility with non-standard ERC20 tokens	28



 Low Severity Issues	29
11. Gas inefficiency from redundant token approvals increases user costs	29
12. Off-by-one error prevents utilizing maximum strategy capacity	30
Automated Tests	31
Closing Summary & Disclaimer	32



Executive Summary

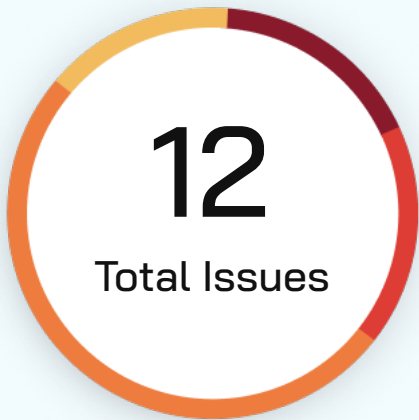
Project Name	CherryOnFinance
Overview	The CherryOn Wagmi Delta Neutral Strategy is a sophisticated DeFi protocol that implements delta-neutral strategies using Wagmi multipools and Silo lending protocols. The system allows users to deposit stable tokens and earn yield while maintaining delta-neutral exposure through automated hedging mechanisms.
Audit Scope	The scope of this Audit was to analyze the CherryOnFinance Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/cherryonfinance/Contracts
Branch	main
Contracts in Scope	CherryOnBalancerDeltaNeutralStrategy/contracts/* CherryOnBeefy&AaveDeltaNeutralStrategy/contracts/* CherryOnBeefy&SiloDeltaNeutralStrategy/contracts/* CherryOnWagmi&AaveDeltaNeutralStrategy/contracts/* CherryOnWagmi&SiloDeltaNeutralStrategy/contracts/*
Commit Hash	ef22a40d9155decb13ac4a489147dabf88f57b61
Language	Solidity
Blockchain	Sonic
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	27th October 2025 - 24 November 2025
Updated Code Received	15th December 2025
Review 2	15th December 2025 -17th December 2025
Fixed In	c4a7cc708108509b00419bf69755707d42fc7890

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	2 (16.66%)
High	2 (16.66%)
Medium	6 (50.0%)
Low	2 (16.66%)
Informational	0 (0.00%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	1	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	2	2	5	2	0



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Precision loss in position estimation reduces capital efficiency and distorts delta neutrality	Critical	Resolved
2	Arithmetic underflow in delta calculation enables denial of service	Critical	Resolved
3	Zero slippage protection in liquidity operations causes protocol insolvency through sandwich attacks	High	Resolved
4	Dangerous slippage bypass for small amounts exposes users to MEV attacks	High	Resolved
5	Excessive rebalance threshold allows dangerous directional exposure	Medium	Acknowledged
6	Missing validation of oracle price staleness allows stale data usage	Medium	Resolved
7	Missing strategy status validation allows deposits into disabled strategies	Medium	Resolved
8	Missing slippage protection allows LP token value manipulation during deposits	Medium	Resolved



Issue No.	Issue Title	Severity	Status
09	Immutable supply cap prevents protocol scaling beyond initial limit	Medium	Resolved
10	Unsafe token transfer prevents compatibility with non-standard ERC20 tokens	Medium	Resolved
11	Gas inefficiency from redundant token approvals increases user costs	Low	Resolved
12	Off-by-one error prevents utilizing maximum strategy capacity	Low	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Critical Severity Issues

Precision loss in position estimation reduces capital efficiency and distorts delta neutrality

Resolved

Path

CherryOnWagmi&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol:562-565
CherryOnWagmi&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol: 532-597
CherryOnBeefy&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol:550-615
CherryOnBeefy&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol

Function Name

`_estimatePosition`

Description

The `_estimatePosition` method calculates the volatile position amount by dividing `amountStableDeposit` by a complex denominator expression before multiplying to derive position sizes. This division-before-multiplication pattern causes integer truncation in Solidity arithmetic where fractional components are permanently discarded.

However, the precision loss creates systematic underestimation of position sizes across all components. The formula computes `amountVolatilePositionInStable` by first dividing the deposit amount by a large denominator containing multiple $1e18$ scaled factors, then using this truncated result to calculate `liquidityVolatileAmount`, `liquidityStableAmount`, `hedgeBorrow`, and `hedgeCollateral`. Each division operation in the chain discards wei-level precision that compounds through dependent calculations. For a 500,000 USDC deposit with typical pool ratios where the denominator expression evaluates to values exceeding $2e18$, the initial division loses approximately 100-1000 wei of precision. When this imprecise intermediate value undergoes subsequent multiplications and divisions to compute hedge and liquidity components, the cumulative error can reach 50-500 USDC worth of undeployed capital. While user LP token ownership remains correct because `getTotalUnderlyingTVL` in line 1329 includes idle balances, the idle funds represent capital inefficiency. More critically, the underestimated hedge positions create imperfect delta neutrality because the actual deployed hedge ratio deviates from the mathematically optimal ratio required to neutralize price exposure.

Consequently, the strategy accumulates idle capital that generates zero yield until subsequent deposits trigger deployment, reducing overall capital efficiency and annualized returns. More significantly, the systematic underestimation of hedge positions means the strategy maintains imperfect delta neutrality with slight directional bias. For a protocol managing 10 million USDC with 0.5% systematic hedge underestimation, approximately 50,000 USDC of directional exposure exists continuously. During a 10% volatile asset price movement, this creates 5,000 USDC of unintended profit or loss, undermining the delta-neutral promise. The idle capital problem compounds over time as precision loss from multiple deposits accumulates, potentially leaving hundreds of thousands in undeployed funds. Users experience lower yields than optimal because idle capital earns no returns from liquidity provision or hedge positions.



Recommendation

We recommend restructuring the calculation to multiply all numerator components before performing final division, thereby preserving maximum precision throughout the computation.

Example -

```
uint256 denominator = ((ratioStable + ratioVolatile) * (SAFE_LTV + delta *  
(1e18 - SAFE_LTV)) ) / (ratioVolatile * SAFE_LTV);  
amountVolatilePositionInStable = (amountStableDeposit * 1e18) / denominator;
```



Arithmetic underflow in delta calculation enables denial of service

Resolved

Path

CherryOnWagmi&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnWagmi&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol

Function Name

`_estimatePosition`

Note

Below issue written in perspective of WAGMI but the same applies to Beefy as well.

Description

The `_calculatePositionDelta` method computes the position delta by subtracting $1e9$ from the square root result without verifying the square root exceeds this threshold. The vulnerable code calculates `uint256 numerator = Babylonian.sqrt((sqrtPriceBX96 *  $1e18$ ) / averageVolatileOpenedPositionPriceX96) - 1e9`, where `sqrtPriceBX96` represents the upper tick boundary of the Uniswap V3 position and `averageVolatileOpenedPositionPriceX96` represents the average entry price tracked by the contract.

However, the contract lacks validation to ensure the square root result exceeds $1e9$ before performing the subtraction. When `averageVolatileOpenedPositionPriceX96` approaches `sqrtPriceBX96` due to market price movements or user deposits near the position boundary, the ratio $(\text{sqrtPriceBX96} * 1e18) / \text{averageVolatileOpenedPositionPriceX96}$ approaches $1e18$. The square root of values near $1e18$ produces results near $1e9$. Specifically, when the ratio equals $999,999,000,000,999,999$ (representing a 0.0001% price difference), the Babylonian square root yields $999,999,500$, which is 500 wei below the $1e9$ threshold. In Solidity 0.8.x and later, arithmetic operations revert on underflow with panic code `0x11`, causing the subtraction $999,999,500 - 1e9$ to revert the entire transaction.

The vulnerability is triggered through normal contract operations without requiring any malicious manipulation. The contract updates `averageOpenedPositionPrice` on every deposit through the `_updateAverageOpenedPositionInfo` function, which computes a weighted average of the previous average price and the current oracle price. As users deposit while the market price approaches the Uniswap V3 position upper tick boundary, the average price naturally converges toward this boundary. Once the average price reaches within approximately 0.0001% of the upper tick price, any subsequent operation calling `_getDelta` triggers the underflow. Since `_getDelta` is invoked during deposit, withdrawal, and all rebalancing operations, the underflow blocks all critical contract functionality, creating a complete denial of service.

Consequently, the entire contract becomes permanently inoperable when the underflow condition is triggered, preventing all users from depositing, withdrawing, or rebalancing their positions. User funds remain locked in the contract until the Wagmi multipool strategy externally adjusts its position tick ranges, which is outside the control of this contract and its users. For protocols managing millions of dollars in total value locked, this represents a catastrophic failure where user assets become inaccessible for an indefinite period. The contract would require emergency intervention through owner-initiated closure of all positions using the `closeAllPositions` function, which itself may fail if it also invokes `_getDelta` during the withdrawal process. The vulnerability affects all deployment instances of this contract and occurs not through malicious exploitation but through normal market dynamics and contract design.



Likelihood

The likelihood of this vulnerability triggering is MEDIUM-HIGH due to the interaction between the Wagmi multipool's dynamic rebalancing and this contract's calculation logic. While the Wagmi multipool strategy automatically adjusts tick ranges as prices move (using tickSpacingOffset to reposition around the current tick dynamically), critical timing windows exist where the underflow can occur.

Consider a realistic scenario demonstrating how normal contract operation triggers the underflow despite Wagmi's dynamic rebalancing:

Initial State (Block N):

- Wagmi multipool strategy has dynamic positioning configured with:

- tickSpacingOffset: -10 (positions slightly below current tick)
- positionRange: 2000 ticks (~2% price range)
- Current pool tick: 200,000 (corresponds to price \$1,950 per ETH)
- Active position tick range: 198,000 to 200,000
- Lower tick 198,000 → \$1,900 per ETH
- Upper tick 200,000 → \$1,950 per ETH

- averageOpenedPositionPrice: \$1,920 per ETH

- Contract TVL: \$5,000,000

Market Movement Phase (Blocks N+1 to N+500):

- ETH price gradually rallies over 12 hours from \$1,920 to \$1,948
- Pool current tick moves from 200,000 to 204,800
- Multiple users deposit during the rally, totaling \$2,000,000 in new deposits
- Each deposit updates averageOpenedPositionPrice using the weighted average formula
- After deposits, averageOpenedPositionPrice: \$1,940 per ETH
- Wagmi has NOT yet rebalanced - still waiting for sufficient tick movement or keeper trigger

Critical Transitional State (Block N+501):

Sudden 0.5% price spike occurs (market volatility, large trade, or arbitrage)

Price jumps to \$1,949.95 per ETH

Pool current tick now at 204,950

Old Wagmi position boundaries still active: upper tick at 200,000 (\$1,950)

Wagmi rebalancing transaction is being prepared but not yet executed

User attempts to withdraw 10,000 USDC

Underflow Trigger Sequence:

1. Withdrawal calls `_withdraw` → `_rebalance` → `_getDelta` → `_calculatePositionDelta`
2. Contract queries `IWagmiMultipoolStrategy.getPositionsFromStrategy()`
3. Returns OLD position info (hasn't updated yet): `upperTick = 200,000`
4. Upper tick `sqrtPriceX96`: $\text{sqrt}(1950 * 1e6) * 2^{96} / 1e18 \approx 1.10456e24$
5. Average price `sqrtPriceX96`: $\text{sqrt}(1949.9998 * 1e6) * 2^{96} / 1e18 \approx 1.10456e24$
6. Ratio: $(1.10456e24 * 1e18) / 1.10456e24 \approx 999,999,000,000,999,999$
7. Square root: $\text{sqrt}(999,999,000,000,999,999) = 999,999,500$
8. Subtraction: $999,999,500 - 1,000,000,000 = -500$ → ARITHMETIC UNDERFLOW
9. Solidity 0.8.x panic code 0x11 → TRANSACTION REVERTS

Next Block (N+502):

- Wagmi keeper transaction executes rebalancing
- New position centered at tick 204,900 with range 202,900 to 206,900
- New boundaries: \$1,930 to \$1,970



- But contract is already frozen - all operations still revert because:
 - averageOpenedPositionPrice = \$1,940 (hasn't reset)
 - Current price = \$1,949.95
 - Even with new boundaries, average price can still approach boundaries through continued deposits

Impact Duration:

- Contract frozen with \$7,000,000 locked
- All deposits revert (cannot add liquidity)
- All withdrawals revert (cannot exit positions)
- All rebalancing operations revert (cannot adjust hedge)
- Resolution requires: Either:
 1. Emergency closeAllPositions by owner (may also fail if it calls _getDelta)
 2. Significant market price movement away from boundaries (unpredictable timing)
 3. Manual intervention to reset averageOpenedPositionPrice (no such function exists)

Recommendation

We recommend implementing comprehensive validation and circuit breaker mechanisms to prevent the underflow condition. First, add explicit validation immediately after computing the square root to ensure it exceeds $1e9$ before performing subtraction. Implement a require statement such as `uint256 sqrtResult = Babylonian.sqrt((sqrtPriceBX96 * 1e18) / averageVolatileOpenedPositionPriceX96); require(sqrtResult >= 1e9, "Price too close to position boundary")`. This prevents the underflow with a clear error message.

Second, implement boundary proximity validation before invoking `_getDelta` in critical operations. Add a view function that calculates the percentage distance between current market price and position boundaries, then require this distance exceeds a minimum threshold such as 0.1% before allowing deposits, withdrawals, or rebalancing. This creates a safety buffer preventing operations when the contract approaches dangerous states.



High Severity Issues

Zero slippage protection in liquidity operations causes protocol insolvency through sandwich attacks

Resolved

Path

CherryOnWagmi&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnWagmi&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol

Description

The `_addLiquidityPosition` method calls `liquidityInfo.multipool.deposit` with hardcoded minimum output amounts of zero in the fourth parameter. Similarly, the `_removeLiquidityPosition` method calls `liquidityInfo.multipool.withdraw` with `amount0OutMin` and `amount1OutMin` both set to zero. These functions interact with the Wagmi multipool (or Beefy vault in Beefy variants) to add or remove liquidity from concentrated liquidity positions. The vulnerability exists identically in all four strategy variants: `CherryOnWagmi&SiloDeltaNeutralStrategy`, `CherryOnWagmi&AaveDeltaNeutralStrategy`, `CherryOnBeefy&SiloDeltaNeutralStrategy`, and `CherryOnBeefy&AaveDeltaNeutralStrategy`.

However, the critical architectural flaw is that deposits are owned by the contract, not individual users. When users call `deposit()`, they transfer stable tokens to the contract and strategy LP tokens got calculated based on the calculated share of total value locked even before actual depositing to underlying vault or strategy. The contract then attempts to deploy these funds into underlying liquidity positions through `_addLiquidityPosition()`. Because the contract mints LP tokens to users before verifying successful deployment, and because slippage protection is set to zero, MEV bots can sandwich attack the liquidity deployment transaction without affecting individual user accounting. The contract suffers the losses while users hold claims against the full deposited amount.

When a sandwich attack occurs, the sequence is: (1) User deposits 500,000 USDC and receives LP tokens representing their proportional ownership of total TVL; (2) Contract calls `_addLiquidityPosition()` with zero slippage protection; (3) MEV bot frontruns with large swap manipulating pool price; (4) Contract's liquidity deployment executes at manipulated price, receiving far fewer LP tokens from the underlying vault (e.g., 475,000 USDC worth instead of 500,000 USDC worth); (5) MEV bot backruns to extract profit. The contract now holds underlying positions worth only 475,000 USDC while owing users LP tokens representing 500,000 USDC of value. This 25,000 USDC shortfall creates immediate undercollateralization affecting all existing LP token holders.

The accounting discrepancy compounds with each sandwiched transaction. The contract calculates LP token minting based on `getTotalUnderlyingTVL()` which includes the user's deposit plus all existing positions. However, after the sandwich attack reduces the actual deployed value, subsequent operations use this lower base for calculations. For example, if the contract holds 10 million USDC in user deposits but only 9.5 million USDC in actual underlying positions due to accumulated sandwich losses, the next withdrawal will fail to return the expected amount because insufficient underlying assets exist to satisfy LP token redemptions. The 500,000 USDC deficit represents protocol insolvency where total liabilities (LP tokens) exceed total assets (underlying positions).



Recommendation

We recommend implementing comprehensive slippage protection with the following multi-layered approach:

Calculate expected output amounts before every liquidity operation using current pool reserves and oracle prices. For Wagmi implementations, query `liquidityInfo.multipool.getReserves()` to obtain current pool reserves, compute the expected LP token amount using reserve ratios, then enforce a minimum threshold of 99-99.5% of expected value (allowing 0-0.5% maximum slippage). For Beefy implementations, call `liquidityInfo.vaultConcLiq.previewDeposit()` to get expected output, then enforce similar minimum thresholds.



Dangerous slippage bypass for small amounts exposes users to MEV attacks

Resolved

Path

CherryOnWagmi&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnWagmi&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol

Function Name

`_estimatePosition`

Description

The `sellExactInVolatile` method checks if `swapAmount` is less than `1e18` (representing 1 ETH) and sets `minAmount` to zero for amounts below this threshold, bypassing all slippage protection. Similarly, the `buyExactOutVolatile` method sets `minAmount` to `type(uint64).max` for amounts less than `1e18`, effectively disabling the maximum input constraint.

However, the threshold of `1e18` wei represents 1 full ETH, which at current market prices is approximately 2,000 to 3,000 USD. This is not a dust amount but a significant value that warrants slippage protection. The comment or intent appears to treat small amounts as inconsequential dust, but the implementation incorrectly applies this logic to substantial values. During withdrawal operations, users often trigger swaps of residual volatile token balances that fall below 1 ETH but represent hundreds or thousands of dollars. MEV bots can sandwich attack these unprotected swaps, extracting 5-10% of the swap value. For a 0.8 ETH swap worth approximately 2,000 USD, an attacker could extract 100 to 200 USD through price manipulation without resistance from slippage constraints.

Consequently, users experience systematic losses on withdrawal operations where residual volatile token balances are converted to stable tokens. These losses occur in addition to any protocol fees and reduce the net value users receive. The vulnerability affects every user withdrawal that does not perfectly align with position ratios, which represents the majority of withdrawals in practice. Cumulative losses across all users can reach tens of thousands of dollars monthly depending on protocol volume. The issue particularly impacts smaller users whose entire withdrawal amounts may fall below the 1 ETH threshold, subjecting their full withdrawal value to unprotected swaps.

Recommendation

We recommend replacing the absolute token amount threshold with a USD-value threshold using oracle prices to determine true dust amounts. Calculate the USD value of the swap amount by multiplying the volatile amount by the Chainlink oracle price, then only disable slippage protection for swaps valued below a minimal threshold such as 10 USD. For amounts above this threshold, maintain slippage protection by calculating expected output based on oracle prices and enforcing a minimum acceptable output. Implement this check as `uint256 volatileValueInStable = getStableAmountForVolatile(volatileAmount)` followed by conditional logic that applies standard slippage protection unless the USD value is genuinely negligible.



Medium Severity Issues

Excessive rebalance threshold allows dangerous directional exposure

Acknowledged

Path

CherryOnWagmi&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnWagmi&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol

Description

The contract defines REBALANCE_THRESHOLD with an initial value of 4500 basis points, representing 45%. This threshold determines when the checkRebalanceNecessity method in lines 1475-1497 triggers rebalancing by comparing the difference between actual and required hedge amounts to this percentage of the current hedge borrow amount.

However, a 45% threshold is excessively permissive for a delta-neutral strategy that aims to maintain zero directional exposure to price movements. Delta-neutral strategies in professional DeFi protocols typically use thresholds between 2% and 5% to ensure positions remain truly neutral. A 45% threshold allows the position to accumulate massive directional exposure before rebalancing triggers. For a protocol managing 10 million USD in total value locked with a 45% delta deviation, the position has 4.5 million USD of directional exposure to the volatile asset. If the volatile asset price moves 10% adversely, the position loses 450,000 USD, which directly reduces user returns and undermines the delta-neutral value proposition.

Consequently, users experience significant losses during periods of price volatility when the position carries large directional exposure. The strategy fails to deliver the market-neutral returns that users expect and that the protocol advertises. Users who selected this strategy specifically to avoid price risk find themselves exposed to substantial price movements. The excessive threshold also creates arbitrage opportunities where sophisticated users can predict when rebalancing will occur and trade against the protocol. Over time, these losses and the failure to maintain neutrality erode user confidence and total value locked as users migrate to better-managed competing protocols.

Recommendation

We recommend reducing the rebalance threshold to between 200 and 500 basis points (2% to 5%) to maintain genuine delta neutrality. Implement a dynamic threshold mechanism that adjusts based on the hedge position health factor, using tighter thresholds when the position is healthy and only relaxing to slightly higher thresholds under stress conditions. For example, use a 2% threshold when the health factor exceeds 2.0, a 3% threshold when the health factor is between 1.5 and 2.0, and a maximum 5% threshold when the health factor falls below 1.5. This ensures the strategy maintains its delta-neutral characteristics while providing flexibility during extreme market conditions.



Missing validation of oracle price staleness allows stale data usage

Resolved

Path

CherryOnWagmi&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnWagmi&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol

Function Name

`_getPriceOracle`

Description

The `_getPriceOracle` method queries Chainlink oracle price feeds and validates that the price update timestamp is recent by comparing `block.timestamp - updatedAt` to `maxOracleAge`. The method reverts with `ERROR_CODE.ORACLE_OUTDATED_DATA` if the price is stale.

However, the method does not validate that the returned price value is positive and non-zero, nor does it verify that the oracle round is complete. Chainlink oracles can return zero or negative prices during oracle failures, incomplete rounds, or when the oracle is paused. The method also does not check the `answeredInRound` value against `roundId` to ensure the round is complete, which Chainlink documentation recommends for production systems. During oracle failures or extreme market conditions, using stale or invalid price data leads to incorrect position calculations. The contract could compute dramatically wrong deposit amounts, withdrawal values, or rebalancing parameters based on zero or stale prices.

Consequently, users could receive far fewer LP tokens than their deposits warrant if a zero price causes incorrect calculations, or the contract could calculate incorrect rebalancing needs leading to position mismanagement. Oracle failures are rare but do occur, as evidenced by several historical incidents in DeFi protocols. When they occur, the impact is severe because all price-dependent operations use incorrect values. A zero price for the volatile asset would cause division by zero errors or massive overestimation of amounts needed. An outdated price during high volatility could cause the contract to rebalance based on prices that are significantly different from current market prices, leading to immediate losses.

Recommendation

We recommend enhancing the oracle validation to include multiple safety checks. Verify that the returned price is greater than zero before using it in calculations. Compare the `answeredInRound` value to `roundId` and revert if they do not match, indicating an incomplete round. Consider implementing a circuit breaker mechanism that pauses the contract if oracle prices deviate more than a reasonable threshold from recent historical prices, indicating a potential oracle malfunction. Store the last valid price as a reference point and validate that new prices are within a reasonable range, such as plus or minus 50% of the previous price, to catch oracle errors before they propagate through calculations.



Missing strategy status validation allows deposits into disabled strategies

Resolved

Path

CherryOnBalancerDeltaNeutralStrategy/contracts/DeltaNeutralStrategyBalancer.sol:287-310

Function Name

`_deposit`

Description

The `_deposit` method iterates through the `_depositStrategies` array and calls `deposit` on each underlying strategy without verifying whether those strategies remain enabled. While `setDepositStrategies` validates strategy status at configuration time in line 253, the owner can subsequently disable a strategy through `disableStrategy` in line 232 without automatically updating the `depositStrategies` array.

However, when a strategy is disabled, it remains in the `depositStrategies` array and `_deposit` attempts to deposit funds into it without runtime validation. The method lacks any check like `isStrategyEnabled(_depositStrategies[i].strategy)` before calling `dnsStrategy.deposit`. This affects the public `deposit` method in line 334, `emergencyRebalance` in line 526, and `emergencyReopenAllPositions` in line 546, all of which call `_deposit` with the potentially stale deposit strategies configuration. Depending on the disabled strategy implementation, deposits may revert, fail silently, or exhibit undefined behavior.

Consequently, user deposits fail when the deposit strategies array contains disabled strategies, creating denial of service for deposit functionality until the owner manually reconfigures deposit strategies using `setDepositStrategies`. The timing gap between disabling a strategy and reconfiguring allocations causes all user deposit transactions to revert without clear error messages. Emergency operations like `emergencyRebalance` and `emergencyReopenAllPositions` also fail when attempting to redeploy funds into disabled strategies, undermining emergency response capabilities during security incidents. Protocol operators cannot efficiently respond to compromised strategies because emergency functions depend on manually synchronized configurations rather than querying current strategy states.

Recommendation

We recommend adding runtime validation in the `_deposit` method by inserting `if (!isStrategyEnabled(_depositStrategies[i].strategy)) continue;` at the beginning of the loop in line 296, skipping disabled strategies and accumulating their allocated amounts in `depositAmountLeft` for retention as idle balance, ensuring deposits only target currently enabled strategies regardless of configuration timing.



Missing slippage protection allows LP token value manipulation during deposits

Resolved

Path

CherryOnBalancerDeltaNeutralStrategy/contracts/DeltaNeutralStrategyBalancer.sol:324-369

Function Name

`deposit`

Description

In CherryOnBalancerDeltaNeutralStrategy/contracts/DeltaNeutralStrategyBalancer.sol:324-369, the deposit method accepts stable token deposits and mints LP tokens based on the ratio $lpAmount = (totalSupply * stableAmount) / getTotalUnderlyingTVL()$ in line 348. The method accepts only recipient and stableAmount parameters without any minimum LP token output requirement.

However, the LP token calculation depends on `getTotalUnderlyingTVL` which fluctuates between transaction submission and execution due to price movements in underlying strategies, sandwich attacks, or frontrunning deposits that dilute the exchange rate. Since line 351 validates only `lpAmount > 0` without enforcing minimum thresholds, users have no protection against unfavorable exchange rates during execution. An attacker can frontrun deposits to inflate total supply, causing victims to receive fewer LP tokens per stable token deposited.

Consequently, users may receive significantly fewer LP tokens than anticipated when submitting deposits, losing value through exchange rate manipulation or market volatility. A user expecting 1000 LP tokens for 1000 USDC may receive only 950 LP tokens if TVL increases 5 percent between submission and execution, representing immediate 5 percent value loss. Frontrunning attacks amplify this effect where attackers deliberately manipulate exchange rates to extract value from deposits.

Recommendation

We recommend adding a `minLpAmount` parameter to the deposit method signature and implementing validation requirement(`lpAmount >= minLpAmount, ERROR_CODE.SLIPPAGE_EXCEEDED`), allowing users to specify minimum acceptable LP token output and reverting deposits that fail to meet slippage tolerance thresholds.



Immutable supply cap prevents protocol scaling beyond initial limit

Resolved

Path

CherryOnBalancerDeltaNeutralStrategy/contracts/DeltaNeutralStrategyBalancer.sol:367

Function Name

`deposit`

Description

The deposit method calls `deltaUsd.mint(recipient, lpAmount)` to mint LP tokens. The `DeltaUsdToken` contract deployed in line 53 enforces a maximum total supply of 1 million tokens through its `mint` method, which reverts when the limit is exceeded. While `DeltaUsdToken` includes a `setMaxTotalSupply` method restricted to `onlyOwner`, the owner is the `DeltaNeutralStrategyBalancer` contract itself.

However, the balancer contract provides no wrapper function to invoke `deltaUsd.setMaxTotalSupply`, making the 1 million token supply cap permanently fixed. The owner multisig controls strategy management but cannot modify token parameters because those functions are not exposed. This architectural oversight prevents any adjustment of the supply limit despite the token contract being designed with upgradeability through `setMaxTotalSupply`.

Consequently, the protocol cannot scale beyond 1 million LP tokens regardless of deposit demand, creating an artificial ceiling on total value locked. Once this limit is reached, all deposit transactions revert with "StrategyToken: Max total supply reached", preventing new users from entering and existing users from increasing positions. A successful protocol experiencing growth will quickly encounter denial of service for deposit functionality. The immutability forces either abandoning the contract and deploying a new instance with migration overhead, or accepting permanent deposit functionality loss once the cap is reached.

Recommendation

We recommend adding a wrapper function `setDeltaUsdMaxSupply(uint256 _maxTotalSupply) external onlyOwner { deltaUsd.setMaxTotalSupply(_maxTotalSupply); }` to the `DeltaNeutralStrategyBalancer` contract, allowing the owner multisig to adjust the LP token supply cap for protocol scaling while maintaining access control through existing ownership structure.



Unsafe token transfer prevents compatibility with non-standard ERC20 tokens

Resolved

Path

CherryOnBalancerDeltaNeutralStrategy/contracts/DeltaNeutralStrategyBalancer.sol

Function Name

`deposit`

Description

The contract imports SafeERC20 library but uses unsafe `transferFrom` and `transfer` methods at lines 358 and 461 instead of their safe counterparts. This causes incompatibility with non-standard ERC20 tokens that may return boolean values from transfer operations, resulting in successful transaction execution despite unsuccessful transfers.

Consequently, The protocol cannot support USDT or similar non-standard tokens as stable tokens, excluding the largest stablecoin by market capitalization and limiting protocol adoption.

Recommendation

We recommend replacing `stableToken.transferFrom(msg.sender, address(this), stableAmount)` at line 358 with `stableToken.safeTransferFrom(msg.sender, address(this), stableAmount)` and `stableToken.transfer(recipient, amount)` at line 461 with `stableToken.safeTransfer(recipient, amount)` using the already imported SafeERC20 library.



Low Severity Issues

Gas inefficiency from redundant token approvals increases user costs

Resolved

Path

CherryOnWagmi&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnWagmi&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&SiloDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol
CherryOnBeefy&AaveDeltaNeutralStrategy/contracts/CherryOnDeltaNeutralStrategy.sol

Description

The `_maxApproveIfNecessary` method checks if the current allowance is below `type(uint128).max` and renews the approval to `type(uint256).max` if needed. This method is called before every swap and liquidity operation in different part of the code.

However, the initial `_maxApproveAll` method sets maximum approvals during contract construction, and these approvals should rarely decrease below the threshold. The frequent checking of allowances and conditional re-approval adds gas costs to every operation. Each allowance call costs approximately 2,600 gas as an external call to the token contract, and the conditional logic adds additional gas overhead. For operations that execute hundreds of times during normal protocol operation, this represents thousands of dollars in cumulative unnecessary gas costs to users. The pattern is particularly inefficient because ERC20 approvals to `type(uint256).max` should never decrease through normal usage.

Consequently, users pay higher gas costs on every deposit, withdrawal, and rebalance operation due to the redundant allowance checking and approval refreshing. Over thousands of transactions, this gas inefficiency accumulates to substantial costs that reduce user returns. While individual transaction overhead may only be a few thousand gas, the cumulative effect across all protocol users represents meaningful economic value. The inefficiency also makes the protocol less competitive compared to gas-optimized alternatives, potentially reducing adoption and total value locked.

Recommendation

We recommend removing the redundant approval checks from frequently called operations and relying on the maximum approvals set during contract construction. The `_maxApproveIfNecessary` method should only be called in the flash loan callback where external contracts might consume approval allowances, not before standard swap and liquidity operations. If approval checking is deemed necessary for safety, implement a caching mechanism that stores whether approvals have been verified in a bitmap or mapping, avoiding repeated external calls to check allowances. This reduces gas costs while maintaining any desired safety properties.



Off-by-one error prevents utilizing maximum strategy capacity

Resolved

Path

CherryOnBalancerDeltaNeutralStrategy/contracts/DeltaNeutralStrategyBalancer.sol

Function Name

`setDepositStrategies`

Description

In CherryOnBalancerDeltaNeutralStrategy/contracts/DeltaNeutralStrategyBalancer.sol:243, the `setDepositStrategies` method validates that `strategyNum < MAX_STRATEGIES_NUM` before accepting deposit strategy configurations. However, in line 174, the `addStrategy` method uses the same validation `strategies.length < MAX_STRATEGIES_NUM` before adding strategies. Since `MAX_STRATEGIES_NUM` is defined as 10 in the Constants contract, both methods use strict inequality, permitting a maximum of 9 strategies rather than 10.

However, the strict inequality creates an off-by-one error where the maximum capacity appears to be 10 based on the constant name but the actual enforceable limit is 9. When exactly 9 strategies exist in the strategies array, attempting to add a tenth strategy through `addStrategy` reverts because `9 < 10` evaluates to true but the next call would fail with `10 < 10` evaluating to false. Similarly, configuring 10 deposit strategies fails in `setDepositStrategies` even though the constant suggests 10 should be the maximum. This inconsistency wastes one strategy slot across the protocol.

Consequently, the contract cannot utilize its advertised maximum capacity, artificially limiting strategy diversity and capital allocation flexibility. Protocol operators expecting to configure 10 strategies based on the constant name will encounter unexpected reverts when attempting to add or configure the tenth strategy. This reduces operational efficiency and creates confusion about the actual strategy limits. The same limitation exists in `DeltaNeutralStrategyBalancerSingleton.sol:264`, affecting both contract variants.

Recommendation

We recommend changing both validation conditions from strict inequality to less-than-or-equal comparison by replacing `< MAX_STRATEGIES_NUM` with `<= MAX_STRATEGIES_NUM` in lines 174 and 243 of `DeltaNeutralStrategyBalancer.sol`, and applying the same correction to `DeltaNeutralStrategyBalancerSingleton.sol:197` and line 264, ensuring the maximum capacity matches the constant name and allowing exactly 10 strategies as intended.



Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of the CherryOnFinance Protocol. We performed our audit according to the procedure described above.

Two critical issues, Two high severity, Six medium severity, while only two low severity were found. One issue was acknowledged, and the remaining were resolved by the team

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

February 2026

For



CHERRY
ON FINANCE



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com